

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL»
по дисциплине «Проектирование и реализация баз данных»

Обучающийся Моломожнов Никита Владимирович

Факультет прикладной информатики

Группа K3240

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

Ментор: Белов Александр

Санкт-Петербург

2025/2026

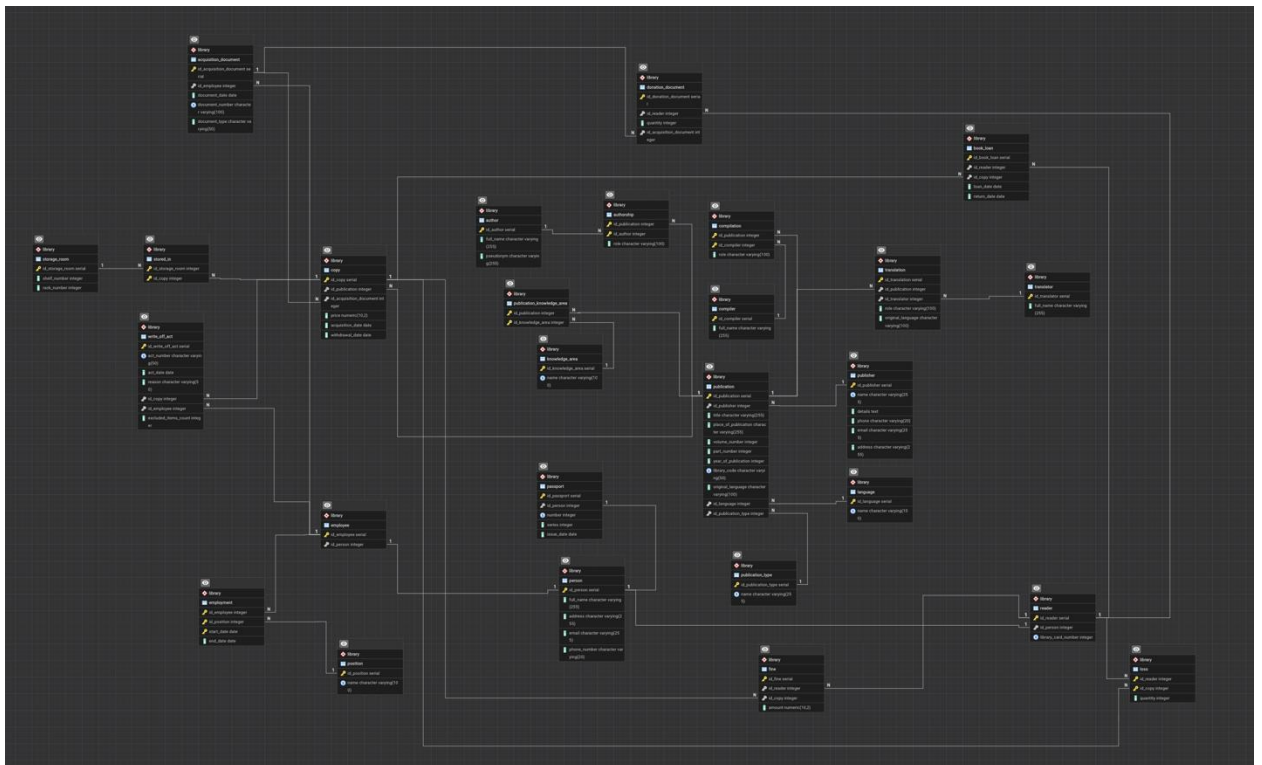
Цель работы:

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры.
2. Создать триггеры для индивидуальной БД согласно варианту. Вариант 2.1 — 7 триггеров по ИЗ.

Схема базы данных (ЛР 3).



Выполнение:

Подключение к БД и установка схемы:

```
psql -U postgres -d "LibraryDB"  
SET search_path TO library;
```

```
Last login: Mon May  4 15:44:33 on ttys000  
nikitamolomozhnov@MacBook-Air-Nikita-9 ~ % psql -U postgres -d "LibraryDB"  
psql (14.19 (Homebrew), server 18.3 (Postgres.app))  
WARNING: psql major version 14, server major version 18.  
Some psql features might not work.  
Type "help" for help.  
  
LibraryDB=# SET search_path TO library;  
SET
```

Процедуры

Процедура 1. Проверка наличия экземпляров книги по названию

```
CREATE OR REPLACE PROCEDURE check_book_availability(  
    IN p_title VARCHAR,  
    INOUT p_count INT DEFAULT 0  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    SELECT COUNT(c.id_copy) INTO p_count  
    FROM copy c  
    JOIN publication p ON c.id_publication = p.id_publication  
    WHERE p.title ILIKE '%' || p_title || '%'  
    AND c.withdrawal_date IS NULL;  
  
    IF p_count = 0 THEN  
        RAISE NOTICE 'Книга % не найдена в активном фонде.', p_title;  
    ELSE  
        RAISE NOTICE 'Книга %: % экз. в наличии.', p_title, p_count;  
    END IF;  
END;  
$$;
```

```
CREATE PROCEDURE  
[LibraryDB=# CALL check_book_availability('War and Peace', 0);  
NOTICE: Книга War and Peace: 503 экз. в наличии.  
p_count  
-----  
503  
(1 row)  
  
[LibraryDB=# CALL check_book_availability('Несуществующая книга', 0);  
NOTICE: Книга Несуществующая книга не найдена в активном фонде.  
p_count  
-----  
0  
(1 row)
```

создание процедуры и вызовы CALL check_book_availability

Процедура 2. Добавление новой книги с экземпляром

```
CREATE OR REPLACE PROCEDURE add_new_book_full(
    IN p_title VARCHAR,
    IN p_publisher_name VARCHAR,
    IN p_place VARCHAR,
    IN p_year INT,
    IN p_library_code VARCHAR,
    IN p_original_lang VARCHAR,
    IN p_lang_name VARCHAR,
    IN p_type_name VARCHAR,
    IN p_price DECIMAL,
    IN p_employee_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_pub_id INT;
    v_lang_id INT;
    v_type_id INT;
    v_doc_id INT;
    v_new_pub_id INT;
BEGIN
    SELECT id_publisher INTO v_pub_id FROM publisher WHERE name = p_publisher_name;
    IF NOT FOUND THEN
        INSERT INTO publisher(name, address) VALUES (p_publisher_name, 'Не указан')
        RETURNING id_publisher INTO v_pub_id;
    END IF;

    SELECT id_language INTO v_lang_id FROM language WHERE name = p_lang_name;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Язык % не найден.', p_lang_name;
    END IF;

    SELECT id_publication_type INTO v_type_id FROM publication_type WHERE name = p_type_name;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Тип % не найден.', p_type_name;
    END IF;

    SELECT id_acquisition_document INTO v_doc_id FROM acquisition_document ORDER BY
    id_acquisition_document DESC LIMIT 1;
    IF NOT FOUND THEN
        INSERT INTO acquisition_document (id_employee, document_date, document_number,
        document_type)
        VALUES (p_employee_id, CURRENT_DATE, 'DOC-' || TO_CHAR(CURRENT_DATE,
        'YYYYMMDD'), 'накладная')
        RETURNING id_acquisition_document INTO v_doc_id;
    END IF;

    INSERT INTO publication (title, id_publisher, place_of_publication, year_of_publication, library_code,
    original_language, id_language, id_publication_type)
    VALUES (p_title, v_pub_id, p_place, p_year, p_library_code, p_original_lang, v_lang_id, v_type_id)
    RETURNING id_publication INTO v_new_pub_id;

    INSERT INTO copy (id_publication, id_acquisition_document, price, acquisition_date)
    VALUES (v_new_pub_id, v_doc_id, p_price, CURRENT_DATE);

    RAISE NOTICE 'Книга % добавлена (id: %).', p_title, v_new_pub_id;
END;
$;
```

```

CREATE PROCEDURE
LibraryDB=# CALL add_new_book_full(
    'Clean Code', 'Pearson', 'Boston', 2008, 'BBK32.973-999',
    'English', 'English', 'Monograph', 3500.00, 1
);
NOTICE: Книга Clean Code добавлена (id: 16).
CALL
LibraryDB=# SELECT p.title, p.library_code, c.price
FROM publication p
JOIN copy c ON p.id_publication = c.id_publication
WHERE p.library_code = 'BBK32.973-999';
    title      | library_code | price
-----+-----+-----
Clean Code | BBK32.973-999 | 3500.00
(1 row)

```

создание процедуры и вызов CALL add_new_book_full и проверка результата

Процедура 3. Регистрация читателя с проверкой дубликата

```

CREATE OR REPLACE PROCEDURE register_reader(
    IN p_full_name VARCHAR,
    IN p_address VARCHAR,
    IN p_email VARCHAR,
    IN p_phone VARCHAR,
    IN p_passport_series INT,
    IN p_passport_number INT,
    IN p_issue_date DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_person_id INT;
    v_card INT;
BEGIN
    IF EXISTS (SELECT 1 FROM passport WHERE series = p_passport_series AND number =
p_passport_number) THEN
        RAISE EXCEPTION 'Паспорт серии % номер % уже существует.', p_passport_series,
p_passport_number;
    END IF;

    INSERT INTO person (full_name, address, email, phone_number)
    VALUES (p_full_name, p_address, p_email, p_phone)
    RETURNING id_person INTO v_person_id;

    INSERT INTO passport (id_person, series, number, issue_date)
    VALUES (v_person_id, p_passport_series, p_passport_number, p_issue_date);

    SELECT COALESCE(MAX(library_card_number), 100000) + 1 INTO v_card FROM reader;

    INSERT INTO reader (id_person, library_card_number)
    VALUES (v_person_id, v_card);

    RAISE NOTICE 'Читатель % зарегистрирован. Билет: %', p_full_name, v_card;
END;
$$;

```

```

CREATE PROCEDURE
LibraryDB=# CALL register_reader(
'Никита Моломожнов', 'ул. Ломоносова, 9', 'nikita@itmo.ru',
'+79273440029', 4520, 999888, '2025-05-12'
);
NOTICE: Читатель Никита Моломожнов зарегистрирован. Билет: 200207
CALL
LibraryDB=# SELECT r.library_card_number, p.full_name, ps.series, ps.number
FROM reader r
JOIN person p ON r.id_person = p.id_person
JOIN passport ps ON p.id_person = ps.id_person
WHERE ps.series = 4520 AND ps.number = 999888;
 library_card_number | full_name | series | number
-----+-----+-----+-----
(1 row)
      200207 | Никита Моломожнов | 4520 | 999888

LibraryDB=# CALL register_reader(
'Другой Человек', 'ул. Дырая, 1', 'test@mail.ru',
'+79990001122', 4520, 999888, '2021-01-01'
);
ERROR: Паспорт серии 4520 номер 999888 уже существует.
CONTEXT: PL/pgSQL function register_reader(character varying,character varying,character varying,character varying,integer,integer,date) line 7 at RAISE

```

успешная регистрация читателя и ошибка при попытке дубликата паспорта

Триггеры

Триггер 1. Контроль лимита выдачи (не более 3 книг)

Основание: ИЗ — «Читателю не может быть выдано более 3-х книг одновременно».

```

CREATE OR REPLACE FUNCTION trg_fn_max_books()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM 1 FROM book_loan
    WHERE id_reader = NEW.id_reader AND return_date IS NULL
    HAVING COUNT(*) >= 3;

    IF FOUND THEN
        RAISE EXCEPTION 'Читатель % уже имеет 3 книги на руках. Выдача запрещена.',
NEW.id_reader;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_max_books
BEFORE INSERT ON book_loan
FOR EACH ROW EXECUTE FUNCTION trg_fn_max_books();

CREATE FUNCTION
CREATE TRIGGER
LibraryDB=# SELECT COUNT(*) FROM book_loan WHERE id_reader = 1 AND return_date IS NULL;
 count
-----
      2
(1 row)

LibraryDB=# SELECT id_copy FROM copy WHERE withdrawal_date IS NULL ORDER BY id_copy LIMIT 5;
 id_copy
-----
      1
      2
      3
      4
      5
(5 rows)

LibraryDB=# INSERT INTO book_loan (id_reader, id_copy, loan_date) VALUES (1, 1, CURRENT_DATE);
INSERT INTO book_loan (id_reader, id_copy, loan_date) VALUES (1, 3, CURRENT_DATE);
INSERT INTO book_loan (id_reader, id_copy, loan_date) VALUES (1, 5, CURRENT_DATE);
INSERT INTO book_loan (id_reader, id_copy, loan_date) VALUES (1, 2, CURRENT_DATE);
INSERT 0 1
ERROR: Читатель 1 уже имеет 3 книги на руках. Выдача запрещена.
CONTEXT: PL/pgSQL function trg_fn_max_books() line 8 at RAISE
ERROR: Читатель 1 уже имеет 3 книги на руках. Выдача запрещена.
CONTEXT: PL/pgSQL function trg_fn_max_books() line 8 at RAISE
ERROR: Читатель 1 уже имеет 3 книги на руках. Выдача запрещена.
CONTEXT: PL/pgSQL function trg_fn_max_books() line 8 at RAISE

```

Триггер 2. Расчёт штрафа при просрочке

Основание: ИЗ — «В случае просрочки читателю назначается денежный штраф».

```

CREATE OR REPLACE FUNCTION trg_fn_fine_calc()
RETURNS TRIGGER AS $$
DECLARE
    v_due DATE;
    v_fine DECIMAL(10,2);
BEGIN
    v_due := OLD.loan_date + INTERVAL '10 days';

    IF NEW.return_date IS NOT NULL AND NEW.return_date > v_due THEN
        v_fine := (NEW.return_date - v_due) * 10.00;
        INSERT INTO fine (id_reader, id_copy, amount)
        VALUES (OLD.id_reader, OLD.id_copy, v_fine);
        RAISE NOTICE 'Просрочка. Начислен штраф: %.2f руб.', v_fine;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_fine_calc
AFTER UPDATE OF return_date ON book_loan
FOR EACH ROW
WHEN (OLD.return_date IS NULL AND NEW.return_date IS NOT NULL)
EXECUTE FUNCTION trg_fn_fine_calc();

```

```

CREATE FUNCTION
CREATE TRIGGER
LibraryDB=# SELECT id_book_loan, id_reader, loan_date, return_date FROM book_loan WHERE return_date IS NULL LIMIT 5;
 id_book_loan | id_reader | loan_date | return_date
-----
          3 |         2 | 2024-02-01 |
        1202 |         1 | 2026-05-10 |
           9 |         1 | 2024-01-15 |
          10 |         2 | 2024-02-01 |
          11 |         3 | 2024-03-01 |
(5 rows)

LibraryDB=# UPDATE book_loan SET return_date = loan_date + INTERVAL '15 days' WHERE id_book_loan = 1;
UPDATE 1
LibraryDB=# SELECT * FROM fine ORDER BY id_fine DESC LIMIT 3;
 id_fine | id_reader | id_copy | amount
-----
        13 |         2 |         9 | 9504.00
        12 |         1 |        17 | 9048.00
        11 |         1 |        14 | 9708.00
(3 rows)

```

начисление штрафа при возврате с просрочкой

Триггер 3. Логирование изменений читателей

Основание: Пример 2 методички — аудит изменений данных

```

CREATE TABLE IF NOT EXISTS readers_log (
    log_id SERIAL PRIMARY KEY,
    action_type VARCHAR(10),
    reader_id INT,
    changed_on TIMESTAMP DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION trg_fn_log_readers()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO readers_log (action_type, reader_id) VALUES ('INSERT', NEW.id_reader);
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO readers_log (action_type, reader_id) VALUES ('UPDATE', NEW.id_reader);
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO readers_log (action_type, reader_id) VALUES ('DELETE', OLD.id_reader);
        RETURN OLD;
    END IF;
END;

```



```

END IF;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_readers
AFTER INSERT OR UPDATE OR DELETE ON reader
FOR EACH ROW EXECUTE FUNCTION trg_fn_log_readers();

LibraryDB=# INSERT INTO person (full_name, address, email, phone_number)
VALUES ('Некий Некто', 'ул. Какаято, 1', 'nekii@test.ru', '+79292300001')
RETURNING id_person;
 id_person
-----
      208
(1 row)

INSERT 0 1
LibraryDB=# INSERT INTO reader (id_person, library_card_number) VALUES (10, 999990);
UPDATE reader SET library_card_number = 999991 WHERE library_card_number = 999990;
DELETE FROM reader WHERE library_card_number = 999991;
ERROR: insert or update on table "reader" violates foreign key constraint "reader_id_person_fkey"
DETAIL: Key (id_person)=(10) is not present in table "person".
UPDATE 0
DELETE 0
LibraryDB=# SELECT * FROM readers_log;
 log_id | action_type | reader_id | changed_on
-----+-----+-----+-----
(0 rows)

LibraryDB=# INSERT INTO reader (id_person, library_card_number) VALUES (208, 999990);
UPDATE reader SET library_card_number = 999991 WHERE library_card_number = 999990;
DELETE FROM reader WHERE library_card_number = 999991;
SELECT * FROM readers_log;
INSERT 0 1
UPDATE 1
DELETE 1
 log_id | action_type | reader_id | changed_on
-----+-----+-----+-----
      1 | INSERT      |      208 | 2026-05-10 22:01:16.202239
      2 | UPDATE      |      208 | 2026-05-10 22:01:16.206952
      3 | DELETE      |      208 | 2026-05-10 22:01:16.207648
(3 rows)

```

записи в readers_log после INSERT, UPDATE, DELETE

Триггер 4. Запрет удаления издания с экземплярами

Основание: ИЗ — учёт выбытия документов, защита ссылочной целостности.

```

CREATE OR REPLACE FUNCTION trg_fn_prevent_delete()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM copy WHERE id_publication = OLD.id_publication) THEN
        RAISE EXCEPTION 'Нельзя удалить издание №% — есть экземпляры.', OLD.id_publication;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_prevent_delete
BEFORE DELETE ON publication
FOR EACH ROW EXECUTE FUNCTION trg_fn_prevent_delete();

```



```

CREATE FUNCTION
CREATE TRIGGER
LibraryDB=# SELECT id_publication, title FROM publication WHERE id_publication = 1;
DELETE FROM publication WHERE id_publication = 1;
 id_publication | title
-----+-----
              1 | War and Peace
(1 row)

ERROR:  Невозможно удалить издание №1 — есть экземпляры.
CONTEXT:  PL/pgSQL function trg_fn_prevent_delete() line 4 at RAISE
LibraryDB=# INSERT INTO publication (title, id_publisher, place_of_publication, year_of_publication, library_code, original_language, id_language, id_publication_type)
VALUES ('Тестовая книга', 1, 'СПб', 2025, 'TST-001', 'Russian', 1, 1)
RETURNING id_publication;
 id_publication
-----
              17
(1 row)

INSERT 0 1
LibraryDB=# DELETE FROM publication WHERE id_publication = 17;
SELECT * FROM publication WHERE id_publication = 17;
DELETE 1
 id_publication | id_publisher | title | place_of_publication | volume_number | part_number | year_of_publication | library_code | original_language | id_language | id_publication_type
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

```

ошибка при удалении издания с экземплярами и успешное удаление издания без экземпляров

Триггер 5. Контроль дат выдачи и возврата

Основание: ИЗ — «Книги выдаются на срок не более 10 дней».

```

CREATE OR REPLACE FUNCTION trg_fn_check_dates()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.loan_date > CURRENT_DATE THEN
        RAISE EXCEPTION 'Дата выдачи не может быть в будущем.';
    END IF;
    IF NEW.return_date IS NOT NULL AND NEW.return_date < NEW.loan_date THEN
        RAISE EXCEPTION 'Дата возврата не может быть раньше даты выдачи.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_check_dates
BEFORE INSERT OR UPDATE ON book_loan
FOR EACH ROW EXECUTE FUNCTION trg_fn_check_dates();

```

```

CREATE FUNCTION
CREATE TRIGGER
LibraryDB=# INSERT INTO book_loan (id_reader, id_copy, loan_date) VALUES (2, 1, '2027-01-01');
ERROR:  Дата выдачи не может быть в будущем.
CONTEXT:  PL/pgSQL function trg_fn_check_dates() line 4 at RAISE
LibraryDB=# INSERT INTO book_loan (id_reader, id_copy, loan_date, return_date) VALUES (2, 2, '2025-01-01', '2024-12-01');
ERROR:  Дата возврата не может быть раньше даты выдачи.
CONTEXT:  PL/pgSQL function trg_fn_check_dates() line 7 at RAISE

```

ошибка «дата выдачи в будущем» и ошибка «дата возврата раньше даты выдачи»

Триггер 6. Автосписание при установке даты изъятия

Основание: ИЗ — «При выбытии документов оформляется акт о списании».

```

CREATE OR REPLACE FUNCTION trg_fn_auto_writeoff()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.withdrawal_date IS NOT NULL AND OLD.withdrawal_date IS NULL THEN
        INSERT INTO write_off_act (act_number, act_date, reason, id_copy, id_employee,
excluded_items_count)
        VALUES ('WO-' || TO_CHAR(CURRENT_DATE, 'YYYYMMDD'), CURRENT_DATE, 'Ветхость',
NEW.id_copy, 1, 1);
        RAISE NOTICE 'Экземпляр №% списан. Создан акт.', NEW.id_copy;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER trg_auto_writeoff
AFTER UPDATE OF withdrawal_date ON copy
FOR EACH ROW EXECUTE FUNCTION trg_fn_auto_writeoff();
```

```
CREATE FUNCTION
CREATE TRIGGER
LibraryDB=# SELECT id_copy, withdrawal_date FROM copy WHERE withdrawal_date IS NULL LIMIT 3;
 id_copy | withdrawal_date
-----+-----
      1 | 
      2 | 
      3 | 
(3 rows)

LibraryDB=# UPDATE copy SET withdrawal_date = CURRENT_DATE WHERE id_copy = 1;
NOTICE: Экземпляр №1 списан. Создан акт.
UPDATE 1
LibraryDB=# SELECT * FROM write_off_act ORDER BY id_write_off_act DESC LIMIT 2;
 id_write_off_act | act_number | act_date | reason | id_copy | id_employee | excluded_items_count
-----+-----+-----+-----+-----+-----+-----
              2 | WO-20260510 | 2026-05-10 | Ветхость |      1 |          1 |                  1
              1 | WO-2024-001 | 2024-01-10 | Ветхость |      9 |          1 |                  1
(2 rows)
```

автоматическое создание акта списания

Триггер 7. Запрет повторной выдачи экземпляра

Основание: ИЗ — экземпляр не может находиться в двух местах одновременно.

```
CREATE OR REPLACE FUNCTION trg_fn_no_double_loan()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM book_loan
        WHERE id_copy = NEW.id_copy AND return_date IS NULL
    ) THEN
        RAISE EXCEPTION 'Экземпляр №% уже находится на руках.', NEW.id_copy;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_double_loan
BEFORE INSERT ON book_loan
FOR EACH ROW EXECUTE FUNCTION trg_fn_no_double_loan();
```

```
LibraryDB=# INSERT INTO book_loan (id_reader, id_copy, loan_date) VALUES (3, 9, CURRENT_DATE);
ERROR: Экземпляр №9 уже находится на руках.
CONTEXT: PL/pgSQL function trg_fn_no_double_loan() line 7 at RAISE
```

ошибка при попытке выдать занятый экземпляр

Дополнительные баллы - модифицированный триггер контроля времени сотрудника

Основание: Практическое задание 1 Лабораторного практикума (Приложение методички).

```
CREATE TABLE IF NOT EXISTS employee_time (
    id SERIAL PRIMARY KEY,
    employee_id INT REFERENCES employee(id_employee),
    is_out_punch BOOLEAN NOT NULL DEFAULT FALSE,
    punch_time TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION trg_fn_check_time()
RETURNS TRIGGER AS $$
```

```

DECLARE
prev RECORD;
BEGIN
IF NOT EXISTS (SELECT 1 FROM employee WHERE id_employee = NEW.employee_id) THEN
RAISE EXCEPTION 'Сотрудник с ID % не найден.', NEW.employee_id;
END IF;

SELECT * INTO prev FROM employee_time
WHERE employee_id = NEW.employee_id
ORDER BY punch_time DESC, id DESC LIMIT 1;

IF NOT FOUND THEN
IF NEW.is_out_punch THEN
RAISE EXCEPTION 'Первая отметка должна быть приходом.';
END IF;
ELSE
IF prev.is_out_punch = NEW.is_out_punch THEN
RAISE EXCEPTION 'Повторный % запрещён.', CASE WHEN NEW.is_out_punch THEN 'уход'
ELSE 'приход' END;
END IF;
IF NEW.punch_time <= prev.punch_time THEN
RAISE EXCEPTION 'Время должно быть позже предыдущего.';
END IF;
IF NEW.is_out_punch AND (NEW.punch_time - prev.punch_time) > INTERVAL '24 hours' THEN
RAISE EXCEPTION 'Смена не может длиться более 24 часов.';
END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_time
BEFORE INSERT ON employee_time
FOR EACH ROW EXECUTE FUNCTION trg_fn_check_time();

```

Реализованы проверки:

- существование сотрудника;
- первая отметка — только приход;
- запрет двух одинаковых отметок подряд;
- контроль хронологии (время позже предыдущего);
- ограничение смены 24 часами.

```

CREATE FUNCTION
LibraryDB=# CREATE TRIGGER trg_check_time
BEFORE INSERT ON employee_time
FOR EACH ROW EXECUTE FUNCTION trg_fn_check_time();
CREATE TRIGGER
LibraryDB=# SELECT id_employee, id_person FROM employee;
 id_employee | id_person
-----+-----
1 | 5
(1 row)

LibraryDB=# INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (1, FALSE, '2026-05-10 09:00:00');
INSERT 0 1
LibraryDB=# INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (1, TRUE, '2026-05-10 18:00:00');
INSERT 0 1
LibraryDB=# INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (1, TRUE, '2026-05-10 19:00:00');
ERROR: Повторный уход запрещён.
CONTEXT: PL/pgSQL function trg_fn_check_time() line 19 at RAISE
LibraryDB=# INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (1, FALSE, '2026-05-09 09:00:00');
ERROR: Время должно быть позже предыдущего.
CONTEXT: PL/pgSQL function trg_fn_check_time() line 22 at RAISE
LibraryDB=# INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (1, FALSE, '2026-05-11 09:00:00');
INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (1, TRUE, '2026-05-13 09:00:00');
INSERT 0 1
ERROR: Смена не может длиться более 24 часов.
CONTEXT: PL/pgSQL function trg_fn_check_time() line 25 at RAISE
LibraryDB=# INSERT INTO employee_time (employee_id, is_out_punch, punch_time) VALUES (999, FALSE, '2026-05-10 09:00:00');
ERROR: Сотрудник с ID 999 не найден.
CONTEXT: PL/pgSQL function trg_fn_check_time() line 6 at RAISE

```

успешные отметки прихода и ухода и ошибки: повторный уход, время раньше предыдущего, смена > 24 часов

Вывод:

В ходе выполнения лабораторной работы №5 были получены практические навыки создания и использования хранимых процедур и триггеров в СУБД PostgreSQL.

Разработаны три хранимые процедуры: `check_book_availability` (проверка наличия экземпляров с возвратом количества через INOUT-параметр), `add_new_book_full` (добавление издания с экземпляром и автоматическим связыванием справочных данных), `register_reader` (регистрация читателя с проверкой уникальности паспорта).

Созданы семь триггеров согласно индивидуальному заданию (вариант 3, БД «Библиотека»): контроль лимита выдачи (не более 3 книг), расчёт штрафа при просрочке, логирование изменений читателей, запрет удаления издания с экземплярами, контроль корректности дат выдачи/возврата, автоматическое создание акта списания при изъятии экземпляра, запрет повторной выдачи занятого экземпляра.

Дополнительно реализован модифицированный триггер контроля времени сотрудника с максимальным учётом узких мест: проверка существования сотрудника, запрет двух одинаковых отметок подряд, контроль хронологии, ограничение длительности смены 24 часами.

Все процедуры и триггеры протестированы в консоли `psql`, работоспособность подтверждена скриншотами. Использование хранимых процедур и триггеров позволяет концентрировать бизнес-логику на стороне СУБД, обеспечивая целостность данных и упрощая разработку клиентских приложений.